

Михаил Разаков

C++ / LLVM Systems Engineer

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

Разрабатываю системные и компиляторные компоненты на C++23/C17/Rust: LLVM-проходы, анализ зависимостей памяти, CFG/SSA, графовые алгоритмы, x86-64 codegen и воспроизводимые тестовые оракулы.

МЦСТ · LLVM LICM

C++ pass, conservative safety gates, semantic before/after harness, -verify-each

1-е из 49 · 104/104

Консервативный PS-form анализ зависимостей памяти и точный оракул

x86-64 pipeline

SSA-подобное IR, liveness, linear scan, iced-x86 и differential validation

ОПЫТ

1 июля — 31 августа 2026

МЦСТ — стажёр по разработке компиляторов

- Реализовал LLVM LICM-pass на C++ с проверками side effects, memory access, loop invariance и speculative safety.
- Проверяю semantic parity baseline/optimized через lli, CHANGE/NO_CHANGE cases и IR validation через -verify-each.

2026

PS-form Memory Dependence Analyzer

- Реализовал консервативный анализ параметрических обращений к памяти; занял 1-е место среди 49 решений, 104/104 тестов.
- Построил точный оракул для малых областей, метаморфные тесты и воспроизводимые WA/TL-контрпримеры.

2026

x86-64 Codegen & Register Allocation Lab

- Спроектировал SSA-подобное IR, анализ живых интервалов, linear scan и сравнение IR-интерпретатора со сгенерированным кодом.

ПРОЕКТЫ

PS-form Memory Dependence Analyzer

1-е место среди 49 решений, единственная оценка 5,0/5,0 и 104/104 официальных тестов; post-selection harness сохраняет воспроизводимые WA/TL-контрпримеры.

x86-64 Codegen & Register Allocation Lab

Измеряемый pipeline с метриками spill/reload, размером кода и воспроизводимым сравнением результатов.

UniversalToolchain / Wist2

Semantic verification и backend-parity checks работают как fail-closed regression gates; verification records привязаны к конкретным revision/run identities.

КОМПЕТЕНЦИИ

C++ / toolchain

C++23, C17, CMake, Linux, clang-tidy, ASan/UBSan, GitHub Actions

Compiler analysis

CFG, dominance, SSA, SCC, dataflow, memory dependence, optimization passes

Code generation

x86-64, iced-x86, liveness, linear scan, IR interpretation, isolated execution

Verification

exact oracles, differential/metamorphic tests, stress tests, counterexample reduction

ОБРАЗОВАНИЕ

Студент программы «Программная инженерия» НИУ ВШЭ

ДОСТИЖЕНИЯ

Программный комитет принял для выступления доклад «Build the Language, Then Make the Abstractions Disappear: Extensible Programming on .NET» об архитектуре UniversalToolchain/Wist2; Торремолинос, Испания.

Москва / удалённо